

Robot Swarm Properties Obtainable by Direct Calculations without Simulation

Jacob Cadavez^a, Daniel Boley^b

^aUniversity of Minnesota,

^bUniversity of Minnesota, boley@umn.edu,

Abstract

We propose a novel way to estimate some properties of the motion of a swarm of robots which does not require carrying out extensive simulations. We leverage the large body of literature on graphs to compute estimates for properties like hitting time, cover time, centrality measures of different regions of an arena, etc. We do not aim to produce accurate physical models of robot behavior, but rather aim to produce estimates of major properties of the robot behavior in the presence of a swarm of robot obstacles using quick closed-form formulas. This would allow the swarm designer to explore a variety of scenarios quickly before advancing to the stage of expensive simulations for more accurate measurements. In this paper, we model the correlated random walk (CRW) of a robot on a discrete arena in discrete time, and then extend this simple model to include multiple robot obstacles. We show how several critical properties for the discrete model can be estimated rapidly using linear algebra tools from spectral graph theory.

Keywords: robot swarms, graph theory, Markov chains, graph Laplacian.

1. Introduction

In swarm robotics, a large group of simple robots cooperates implicitly to collectively perform tasks. The robots in general have limited individual capabilities in terms of sensing, processing power, and inter-agent communications, but the number of robots makes up for the individual limitations.

Robot swarms can be used for many real-world tasks, such as cleaning an area [1], demining, rescuing people, or mowing a lawn, which require complete coverage of the space and where ideally each location should be visited only once to avoid unnecessary repetitions, or tasks that require maintaining surveillance over an area to detect intruders, fires, traffic jams or enemies [2, 3], or to map an unknown area.

Robot swarms are a useful tool for monitoring and exploring different areas with great efficiency due to their complete coverage with minimal repetitions. This shows itself in real-world situations such as: search-and-rescue, uncovering areas with risks of fires, traffic jams, and more. [4].

A task of special interest in the swarm community, due to its diversity of utility real-world applications, is “foraging”, where multiple robots are released in a pre-defined area to search for target objects and bring them back to a centralized location called the nest. Multiple robots could be released to search for targets within a predefined area.

We denote this process as “searching” because robots don’t know where the target locations are. In this paper, we approximately model foraging by a single robot in the presence of multiple roving robot obstacles. The foraging behavior by a swarm can in principle be extrapolated from the estimated behavior of a single robot in the presence of other roving robots. In these situations, it is often necessary for engineers to build robot swarms that will accomplish the given task within a set time constraint. Designers will have to simulate and experiment with multiple variables such as the number of robots and movement behavior in order to fully decide on a system that suits their immediate needs. An important metric that is used to decide the configuration for a swarm mission is the expected search time or ‘hitting time’ of a simulation. This measurement can be very resource and time intensive to obtain. The most accurate method to collect data is to run experiments involving real robots, which is usually infeasible due to its heavy time and resource consumption. Instead, researchers typically use a computer application to run a large number of simulations and perform data analysis on the results to obtain empirical models. However, these trials can become very taxing when scaling to larger arena sizes and may not accurately represent how these swarms may behave in practice, even with ARGoS, an industry accepted software to run such simulations [5].

We propose an alternative approach by using mathematical models to quickly approximate robot swarms’ average hitting times over full-simulation methods. We specifically do *not* aim for physical accuracy, but rather aim to quickly produce approximations using closed-form formulas that are close enough to give a good qualitative measure of the robot behaviors under a variety of scenarios of interest to the designer.

The mathematical computations we propose in this paper are based off a variant of the random walk (RW) model - a movement policy that dictates how each robot moves from step to step. Although very simplistic, the RW model is frequently used for exploration due to its scalability and modest processing requirements [6, 7, 8]. The RW model is a discrete-time stochastic process where successive random steps are used. When the number of possible states is finite or countable and the step taken at each discrete time interval depends only on the current state (or at most a fixed sized history), the process can be described by a discrete-time Markov chain [9]. An important property of random walks is the “first passage time,” or Hitting time (HT), which is the average time to first visit a node while doing a random walk from another node in a given network [10, 11].

An extension of the RW model is the Correlated Random Walk (CRW) model where the probability of each robot’s next step depends on its current orientation and location, or more precisely the current location and immediately preceeding location. The CRW variant has been explored in [12, 13, 14, 15, 6]. Although not the most optimized search method, CRWs are commonly used to model insect behaviors, which is the motivation behind a large fraction of the research in swarms. As real robots are often non-holonomic or require extra steps to rotate in place, CRWs are a better model than ordinary RWs as they take direction into consideration. By defining state as both location and direction and displaying a CRW model’s movement behavior as a graph, one can observe that each state transition is directed. For example, transitioning from a location A to a neighboring location B and then back to A, one ends up at A with different orientation, represented by a different state in the Markov chain. Thus the links in the Markov chain naturally one-way directed. With the Markov property in effect, this allows more dimensionality in swarm modeling

allowing more accurate hitting time calculations instead of not taking direction into account.

Starting with computational methods to estimate probability distributions for solo robots, we aim to extend this to a setting with multiple robots, with or without obstacles. An earlier short version of this paper [16] introduced the basic theoretical setup of a discrete-time-discrete-space computational model, and gave some illustrations on how this corresponded to practical behavior. In this paper, we reprise the theory and give more extensive simulations for the discrete-time-discrete-space computational model, illustrating both the agreement and the limitations of the theoretical model with respect to the behavior in practice.

As there is previously developed work that showcases how computational models can estimate probability distributions of a solo robot, this paper will focus on multi-robot systems of obstacles. In this work, we develop computation models that can rapidly compute estimates of probability distributions of a specific robot among a swarm of other robots assuming each robot follows the same movement policy and acts independently from other robots.

Our aim is to develop and measure empirically models that are able to capture swarm motion behaviors well enough to make global predictions about their performance.

To the authors' knowledge, the spectral analysis of directed graphs has not been applied directly to multi-robot systems or swarm robots. Thus, we aim to leverage some of the methods to rapidly compute aggregate properties of digraphs and to develop a computation model that can rapidly and accurately approximate properties such as the HT in the multi-robot searching process.

The main contributions of this paper are:

1. Show how a model in discrete time and space can be used to elicit certain properties of a robot or a swarm of robots.
2. Leverage recent developments in spectral graph theory for digraphs to obtain a fast method to obtain approximate average hitting times, one of most valuable and foundational properties.
3. Illustrate how the graph representation of the scenarios yields several other useful aggregate properties, all without simulations, even on irregular arenas.

The rest of this paper consists of a section on related work, a section on the basic mathematical constructions mapping from a digraph to its associated Laplacian matrix and thence to properties of individual agents operating in this environment, a section on extrapolating from the single-agent estimates to estimates of multi-agent behaviors, and a final section with some conclusions, limitations and possible future directions.

2. Related Work

An enormous body of work exists on the analysis of random walks on graphs such as estimating hitting times, centrality measures, cover times, and other aggregate properties, mainly for undirected graphs. Many variations on robot swarm models have appeared, but primarily focused on behaviors reliant on communicating with neighboring robots (e.g., [1]). Our study focuses on robots that move independently within an arena to accomplish a task.

There is a long history of study of Markov chain properties that can be obtained directly from the probability transition matrix and matrices derived therefrom, too much to fit in this paper. Only recently has a study of the computational issues for directed graphs or non-reversible Markov chains has been addressed (e.g., [17, 18]). The relation between the undirected graph Laplacian and connectivity of undirected graphs were treated for instance in [19, 20, 21, 22]. The computation of graph connectivity properties based on the graph Laplacian was used in specific applications in expander graphs [23], recommender systems [24], computer vision [25]. All of these were for undirected graphs.

Fewer papers exist discussing directed graphs (digraphs) in limited contexts. For example, a short treatment within a text mostly devoted to the symmetric case is in [26]; the Matrix tree theorem is presented and extended with some results to the directed graph case in [27]; eigenvalue bounds for the Cheeger constant, previously developed for undirected graph, are extended to digraphs by using closely related undirected graphs in [28, 17]. Computing the eigenvalues of the appropriate graph Laplacian matrix is much easier than computing the Cheeger constant itself, so these eigenvalues yield a quick estimate on the overall connectivity of the digraph.

Much fundamental material on random walks, Markov chains, recurrence times, and related topics can be found in these books [29, 30, 9] and survey on random walks [10] and the survey [11] which discusses a wide range of properties of random walks including cover and hitting times, stationary probabilities in both discrete and continuous time.

Another thread of research has produced some theoretical bounds on how many robots are needed to explore a grid using a systematic finite-state algorithm with a collection of robots that have constant memory and limited sensors [31], or can observe all other robots' positions [32], in the context of a cleaning or exploration task.

Some recent papers have addressed the analysis of different exploration strategies for one or more agents over a graph [6, 33]. The papers [34, 35, 36, 37] use spectral properties of the underlying Markov chain to derive bounds on the hitting and cover times. The papers [38, 39] give hitting and cover times for specific graphs (e.g., cycles) or graphs that maximize the cover time, while [40] does the same for a tree. Hitting times for multiple simultaneous random walks are treated in [41, 42]. The paper [43] generalized the fundamental matrix to the fundamental tensor encapsulating betweenness measures.

Many results for hitting time (HT) computations exist. For example, the average time to reach a given node can be computed based on the fundamental matrix associated with the probability transition matrix of the network [44], by treating the target node as an absorbing state. In the paper [45], the authors showed how to obtain average HTs from any node to any other node in a directed graph at once using the asymmetric graph Laplacian.

Obtaining a-priori estimates of hitting and cover times are very useful if one can avoid the expense of long simulations since they allow one to efficiently iterate and adjust the system to optimize performance. Several authors [38, 46, 47, 48, 49] found that the bounds on the cover time vary greatly depending on the starting node, the number of agents, and even the layout of the specific graph. For a two-dimensional $n \times n$ grid (the layout closest to the graphs of interest in this paper), it was found that a small number of independent agents can reduce the cover time propor-

tionally, but beyond $O(\log n)$ agents it can quickly saturate [46, 49]. The papers [41, 42] also address how the cover time varies with the number of agents. These theoretical results suggest that some coordination between the agents, or carefully staging the agents initially in different parts of the domain, would make a big difference. These papers are focused on ordinary walks over an undirected graph and haven’t been adapted into a CRW where the direction plays a factor in movement behavior.

A few papers derive overall estimates of the probabilities of collisions theoretically [50, 51], but these do not involve simulations of robot agents. Given that collisions are hard to model using a memoryless process with transition probabilities fixed in advance, these papers have incorporated an average collision probability fixed upfront (e.g., [52] & references therein).

Another body of papers derives facts about the probability distribution of hitting times, cover times, or mixing times beyond the mean values, some in terms of higher moments or fitting a standard probability distribution [53, 54, 55, 56, 57]. Formulas for the hitting time probability distribution have been obtained using the Laplace and Z transforms, respectfully in [58, 13]. The higher moments are used for network seed-set selection in [59].

The computation of the moments involves the solution of large linear systems involving various forms of the graph Laplacian or probability transition matrix, as in [60, 61] based on the GTH algorithm [62]. Later works were based on a mix of direct and iterative methods [18, 63]. Here we have adopted a purely iterative method, namely restarted GMRES, due to its simplicity and easily available implementations.

Much of the work described so far is theoretical, with no experimental work with real robots or physics-based simulators. There is a separate community of research focussed on real robots and/or simulations to mimic real robot scenarios when real robot experiments are impractical, e.g. with physics simulators such as ARGoS [5].

Many swarm algorithms use random or correlated random walk, because of their simplicity. Some instead synchronize the robot motions through the environment [1], where robots clean a non-convex region using the dirt on the floor as the main means of inter-robot communication and follow a strict protocol to guarantee full coverage.

A recent survey of foraging algorithms [64] analyzes the state of the art in foraging, with special attention to the foundations of swarm research as well as to applications of robot swarms. A specific version of foraging called “Central Space Foraging,” is characterized by having the collection area (goal node) in the center of the space, which is circular. The problem has been studied systematically [65] to analyze the performance of a few different algorithms. Specifically, the robots move randomly, or follow a deterministic path such as a multi-robot Archimedes spiral, or move radially from the center. The comparisons in the paper were done experimentally, but the paper provided theoretical upper bounds for the time needed to complete the task.

Some theoretical results on emergent behaviors in systems of many simple agents with limited memory and limited communication have appeared in [66, 67]. For example, they considers a uniform triangular/hexagonal grid with periodic boundary conditions. Theoretical guarantees are given that congregation or dispersion will naturally arise depending on the setting of a presence of “food.”

3. Proposed Approach for Direct Computations

In the swarm community, the environment used for searching tasks is typically 2D, most often with continuous space. In our approach, instead, we use a discrete space and a correlated random walk (CRW) to model the motion of the robots in discrete time. We generalize the grid representation with a graph, and use methods developed for graphs to estimate the motion behavior of the robots in the swarm. Our objective is not to design controllers for the robots **nor to get exact reproduction of the behavior of physical robots**, but to get quick estimates of swarm properties by direct calculation. This allows one to explore quickly a variety of scenarios, reserving the expensive simulations for scenarios of specific interest. **Previous papers (e.g. [68, 69]) investigated the correspondence between our estimates of the first two moments of the hitting time, the Gamma distribution approximating the distribution of hitting times, and the behavior of physical robots as approximated by simulators such as ARGoS. They found that for single robots the Gamma distribution induced by the first two moments was a pretty good match to the results of ARGoS simulations, after some adjustment for idiosyncrasies of the physical models.**

We show next how to obtain the approximate distribution of the basic property, the hitting time (HT), by a direct calculation using the method of moments, thereby avoiding the expense of running simulations. Analogous properties can be obtained using the non-symmetric graph Laplacian and other matrices derived therefrom [43, 35].

In later sections, an outline of how the simulation works and how the Markov-chain approach can yield values for important moments is given. Throughout this paper, we assume that the graph underlying the Correlated Random Walk is strongly connected, or would be so connected if it were modified to give a non-zero probability to a transition away from the goal node. In other words there is a feasible path of nonzero probability from any node to any other node, when outgoing links from the goalnode are temporarily modified to have non-zero probabilities.

3.1. Computation of HT Mean and Variance

Let $\mathbf{P}$ denote the probability transition matrix for the Markov chain, such that p_{ij} (the entry in row i column j) is the probability that the robot in state i will move to state j at the next time step. We define two vectors: the mean $\mathbf{h}_\mu$ and variance $\mathbf{h}_{\sigma^2}$, whose i -th entry is the mean and variance (respectively) of the Hitting Time from the i -th state to the nest: the number of time steps needed to reach the nest from state i .

Following [44, 68], we partition $\mathbf{P}$ to get $\mathbf{Q}$, the probability transition matrix corresponding to the non-absorbing states (i.e., **all the states other than goal states**), via:

$$\mathbf{P} = \begin{bmatrix} \mathbf{Q} & \mathbf{r} \\ \mathbf{0} & 1 \end{bmatrix}. \quad (1)$$

Here $\mathbf{r}$ is a single vector whose i -th entry is the probability of transitioning from state i to the absorbing state in a single time step. These formulas can be easily generalized to the case of multiple absorbing states where $\mathbf{r}$ is a matrix. The matrix $\mathbf{P}$ is row stochastic since the matrix is non-negative and all the row sums equal 1. So $\mathbf{P}\mathbf{e} = \mathbf{e}$, where $\mathbf{e}$ is

a vector of all ones of appropriate dimension. With some algebra, this implies [68] that $\mathbf{r} = (\mathbf{I} - \mathbf{Q})\mathbf{e}$.

$$\mathbf{Q}\mathbf{e} + \mathbf{r} = \mathbf{e} \quad \implies \quad \mathbf{r} = (\mathbf{I} - \mathbf{Q})\mathbf{e} \quad (2)$$

The probability of reaching the absorbing state on exactly the ℓ -th time step, starting from node i , is $[\mathbf{Q}^{\ell-1}\mathbf{r}]_i$, i.e., the i -th entry of the vector $\mathbf{Q}^{\ell-1}\mathbf{r}$ of these probabilities. It is well known that the k -th uncentered moment of this probability distribution is

$$\text{umoment}(k) = \sum_{\ell=1}^{\infty} \ell^k \mathbf{Q}^{\ell-1}\mathbf{r}. \quad (3)$$

The vector of means is the first moment: $\mathbf{h}_\mu = \text{umoment}(1)$ and the variance is the centered second moment: $\mathbf{h}_{\sigma^2} = \text{umoment}(2) - \mathbf{h}_\mu^2$, where “ $\cdot^2$ ” denotes elementwise squaring.

We define the fundamental matrix $\mathbf{N}$ of $\mathbf{P}$ as [44]:

$$\mathbf{N} \stackrel{\text{def}}{=} (\mathbf{I} - \mathbf{Q})^{-1}. \quad (4)$$

We obtain expressions [68] for the HT mean and variance using the following power series expansions in $\mathbf{Q}$ for $\mathbf{N}$, $\mathbf{N}^2$ and $2\mathbf{N}^3 - \mathbf{N}^2$:

$$\begin{aligned} \text{(a)} \quad \mathbf{N} &= \mathbf{I} + \mathbf{Q} + \mathbf{Q}^2 + \mathbf{Q}^3 + \mathbf{Q}^4 + \dots \\ \text{(b)} \quad \mathbf{N}^2 &= \mathbf{I} + 2\mathbf{Q} + 3\mathbf{Q}^2 + 4\mathbf{Q}^3 + 5\mathbf{Q}^4 + \dots \\ \text{(c)} \quad 2\mathbf{N}^3 - \mathbf{N}^2 &= \mathbf{I} + 4\mathbf{Q} + 9\mathbf{Q}^2 + 16\mathbf{Q}^3 + 25\mathbf{Q}^4 + \dots \end{aligned} \quad (5)$$

Formula (5b) yields the expression for the first moment $\mathbf{h}_\mu$ and formula (5c) yields the expression for the uncentered second moment. These in turn yield the mean and variance: $\mathbf{h}_\mu$ and $\mathbf{h}_{\sigma^2}$:

$$\begin{aligned} \mathbf{h}_\mu &= \mathbf{N}^2\mathbf{r} = \mathbf{N}\mathbf{e} \\ \mathbf{h}_{\sigma^2} &= (2\mathbf{N}^3 - \mathbf{N}^2)\mathbf{r} - \mathbf{h}_\mu^2 = 2\mathbf{N}\mathbf{h}_\mu - \mathbf{h}_\mu - \mathbf{h}_\mu^2. \end{aligned} \quad (6)$$

This gives closed-form expressions for the mean and variance of the hitting time. With these two parameters in hand, one can fit a 2-parameter distribution to the hitting time and use that to estimate quantities such as the probability that the nest will be reached within a given time limit. This is done later in Section 3.3. However we first discuss the process to compute the quantities in Eqs. (6).

3.2. Fast Solution of Linear Systems

Multiplying a vector $\mathbf{v}$ by the matrix $\mathbf{N}$ is equivalent to solving the system of linear equations of the form $(\mathbf{I} - \mathbf{Q})\mathbf{u} = \mathbf{v}$ for the vector $\mathbf{u}$. In this subsection we show how the system of equations involving $\mathbf{I} - \mathbf{Q}$ can be solved efficiently, taking full advantage of the high degree of sparsity in the matrix $\mathbf{I} - \mathbf{Q}$. This system can be efficiently solved for by iterative methods such as Restarted GMRES [70, 71, 72, 63], even if $\mathbf{I} - \mathbf{Q}$ is extremely large (e.g., $100,000 \times 100,000$ or larger), since it is very sparse (at most 6 entries in every row in our current setup).

The system of linear equations involving the matrix $\mathbf{N} = (\mathbf{I} - \mathbf{Q})^{-1}$ derived from a random walk can be transformed into a system enjoying special properties that make it possible to solve using methods that are faster than using a standard in-place elimination algorithm. To take full advantage of the sparsity present in the matrix $\mathbf{Q}$, we would like to use an iterative method such as GMRES which accesses the matrix only through forming matrix-vector products. If an $n \times n$ matrix $\mathbf{A}$ has only m nonzero elements (corresponding to a graph with n vertices and m edges), then the matrix-vector product $\mathbf{A}\mathbf{v}$ costs only nm multiplications, less than n^2 multiplications for a full matrix. The key missing element is to provide a preconditioner to guarantee that an iterative method like GMRES will converge. We accomplish this by embedding $\mathbf{Q}$ into an modified probability transition matrix $\tilde{\mathbf{P}}$ corresponding to a random walk with no transient states:

$$\tilde{\mathbf{P}} \stackrel{\text{def}}{=} \begin{bmatrix} \mathbf{Q} & \mathbf{r} \\ \mathbf{s}^\top & t \end{bmatrix}, \quad (7)$$

such that $\tilde{\mathbf{P}}$ is row stochastic ($\mathbf{P}\mathbf{e} = \mathbf{e}$) and $\mathbf{s} \geq 0$ is not entirely zero. Here $\mathbf{e}$ is the vector of all ones. the matrices $\tilde{\mathbf{P}}$ and $\mathbf{P}$ differ only in the last row. The assumption that the underlying graph is strongly connected guarantees the existence of $\tilde{\mathbf{P}}$ and that this matrix has $\lambda = 1$ as a simple eigenvalue.

The steps required to solve $(\mathbf{I} - \mathbf{Q})\mathbf{u} = \mathbf{v}$ for the vector $\mathbf{u}$ are outlined here, starting with the probability transition matrix $\mathbf{P}$ (eq. (1)). First we have 4 “preprocessing steps” to be done just once. (I)

1. Choose non-negative $\mathbf{s}, t$ such that $\mathbf{s}^\top \mathbf{e} + t = 1$ to form the matrix $\tilde{\mathbf{P}}$ (7).
2. Compute vector of stationary probabilities $\boldsymbol{\pi} = (\pi_1, \dots, \pi_n)^\top \in \mathbb{R}^n$ satisfying
 - (a) $\boldsymbol{\pi} > 0$ and $\boldsymbol{\pi}^\top \mathbf{e} = 1$,
 - (b) $\boldsymbol{\pi}^\top \tilde{\mathbf{P}} = \boldsymbol{\pi}^\top$.

Let $\boldsymbol{\Pi} = \text{diag}(\boldsymbol{\pi}) \in \mathbb{R}^{n \times n}$ denote the diagonal matrix with the stationary probabilities on the diagonal. The existence of such a vector $\boldsymbol{\pi}$ is guaranteed by the fact that the graph underlying the Markov chain represented by $\tilde{\mathbf{P}}$ is strongly connected.

3. Define the Random Walk Laplacian matrix

$$\mathbf{L} \stackrel{\text{def}}{=} \boldsymbol{\Pi} - \boldsymbol{\Pi} \tilde{\mathbf{P}} = \begin{bmatrix} \boldsymbol{\Pi}_1(\mathbf{I} - \mathbf{Q}) & -\mathbf{r} \\ -\mathbf{s}^\top & \pi_n - t \end{bmatrix} \in \mathbb{R}^{n \times n}, \quad (8)$$

where $\boldsymbol{\Pi}_1$ denotes the upper left $(n-1) \times (n-1)$ block of $\boldsymbol{\Pi}$. Here n is the number of states in the Markov chain.

4. Compute the last column of the Moore-Penrose pseudo-inverse $\mathbf{L}^\dagger$:

$$\mathbf{g} \equiv \begin{bmatrix} \mathbf{g}_1 \\ g_n \end{bmatrix} \stackrel{\text{def}}{=} \mathbf{L}^\dagger \begin{bmatrix} \mathbf{0} \\ 1 \end{bmatrix} \in \mathbb{R}^n.$$

The fast computation of $\mathbf{L}^\dagger$ times a vector is discussed below.

Then to compute $\mathbf{u} \stackrel{\text{def}}{=} (\mathbf{I} - \mathbf{Q})^{-1} \mathbf{v}$ for some arbitrary vector $\mathbf{v} \in \mathbb{R}^{n-1}$, we carry out the following computation for each $\mathbf{v}$:

5. Embed the vector $\mathbf{v} \in \mathbb{R}^{n-1}$ in an $n \times n$ system and compute

$$\mathbf{w} \equiv \begin{bmatrix} \mathbf{w}_1 \\ w_n \end{bmatrix} \stackrel{\text{def}}{=} \mathbf{L}^\dagger \mathbf{\Pi} \begin{bmatrix} \mathbf{v} \\ 0 \end{bmatrix} \in \mathbb{R}^n.$$

6. Compute

$$\mathbf{u} = \mathbf{w}_1 - \mathbf{e}w_n + (g_n \mathbf{e} - \mathbf{g}_1) \left(\boldsymbol{\pi}^\top \begin{bmatrix} \mathbf{v} \\ 0 \end{bmatrix} \right) \in \mathbb{R}^{n-1}.$$

We elaborate on each of these steps, based on the theory summarized in [63].

1. The choice for $\mathbf{s}^\top$ is somewhat arbitrary, but choosing $\mathbf{s}$ to make the column sums as equal as possible is a good choice [73]. The existence of an appropriate $\mathbf{s}$ depends on the graph property of being strongly connected.
2. There are many ways to compute the vector of stationary probabilities, but an iterative method that is limited to matrix-vector products, taking full advantage of the inherent sparsity, is restarted subspace iteration [74]. It is based on tracking a subspace of dimension $\ell \geq$ the period of the graph (if any), and has cost of $O((nm\ell + n\ell^2 + \ell^3)/|\log(|\lambda_{\ell+1}|)|)$ FLOPS, where n = number of vertices, m = number of edges and $|\lambda_{\ell+1}| < 1$ is the $\ell + 1$ -th largest eigenvalue of $\tilde{\mathbf{P}}$ in absolute value. The larger the ℓ , the fewer iterations are required, but the cost of each iteration and memory requirement increase. We remark that if one uses $\ell = 1$, this method reduces to the classical power method for eigenvalues, which converges if the graph is aperiodic. Larger values for ℓ will accelerate convergence and allow convergence even for periodic chains.
3. The matrix-vector product $\mathbf{L}^\dagger \mathbf{v}$ is computed by applying an iterative method like GMRES to the invertible matrix $\mathbf{L} + \mathbf{E}$ with $\mathbf{E} = \frac{\mathbf{e}\mathbf{e}^\top}{n}$, $\mathbf{e}$ being a basis for the nullspace of $\mathbf{L}$. It is not necessary to form the matrix $\mathbf{L}$ explicitly, as long as one has a method to compute matrix-vector products with it. This matrix has the property of having a one-dimensional right nullspace equal to its left nullspace: $\mathbf{L}\mathbf{e} = \mathbf{L}^\top \mathbf{e} = \mathbf{0}$. This implies that

$$\mathbf{L}^\dagger = (\mathbf{L} + \mathbf{E})^{-1} - \mathbf{E}, \tag{9}$$

where $\mathbf{E} \stackrel{\text{def}}{=} \mathbf{e}\mathbf{e}^\top/n$ is an $n \times n$ matrix with all entries equal to $1/n$ (verifiable by direct calculation [63]).

- 4 & 5. These expressions are based on the identity (9). Hence the product of the pseudo-inverse with any vector $\mathbf{y}$ is $\mathbf{L}^\dagger \mathbf{y} = (\mathbf{L} + \mathbf{E})^{-1} \mathbf{y} - \mathbf{e} \frac{\mathbf{e}^\top \mathbf{y}}{n}$. Computing the expression $(\mathbf{L} + \mathbf{E})^{-1} \mathbf{y}$ is carried out by solving the system of equations $(\mathbf{L} + \mathbf{E})\mathbf{x} = \mathbf{y}$ via an iterative method like Richardson iteration or GMRES, as outlined in [63].

An iterative method like GMRES or Richardson iteration is guaranteed to converge because the symmetric matrix $(\mathbf{L} + \mathbf{E}) + (\mathbf{L} + \mathbf{E})^\top$ is positive definite [63]. To see this, note that $\mathbf{L}$ is the combinatorial Laplacian to a graph with adjacency matrix $\mathbf{\Pi}\tilde{\mathbf{P}}$, and $(\mathbf{L} + \mathbf{L}^\top)/2$ is the combinatorial Laplacian for a weighted undirected graph with adjacency matrix $(\mathbf{\Pi}\tilde{\mathbf{P}} + \tilde{\mathbf{P}}^\top \mathbf{\Pi})/2$. This implies that $(\mathbf{L} + \mathbf{L}^\top)/2 + \mathbf{E}$ is positive definite, and hence $\mathbf{L} + \mathbf{E}$ satisfies the conditions for guaranteed convergence of Richardson iteration and GMRES [63].

Table 1: Computation costs for different sized arenas.

arena size	matrix $\mathbf{P}$ dimension	number non-zeros	setup time (sec)	GMRES time (sec)
30×30	5767	31223	1.5	0.05
150×150	136807	807863	38.4	5.4
300×300	543607	3236663	44.9	35.6

6. This is based on the following identity for any singular matrix A such that $C \stackrel{\text{def}}{=} A + \alpha \mathbf{u} \mathbf{u}^T$ is invertible. Partition A and $\mathbf{u}$ as follows:

$$A = \begin{bmatrix} A_{11} & \mathbf{a}_{12} \\ \mathbf{a}_{21}^T & a_{nn} \end{bmatrix}, \quad \mathbf{u} = \begin{bmatrix} \mathbf{u}_1 \\ u_n \end{bmatrix}. \quad (10)$$

Assume $\mathbf{u}^T \mathbf{u} = 1$ and $u_n > 0$. Let $B \stackrel{\text{def}}{=} A^\dagger$ be partitioned similarly. Then A_{11}^{-1} exists and can be written in terms of the pseudo-inverse $A^\dagger = B$:

$$A_{11}^{-1} = B_{11} - \frac{1}{u_n} \mathbf{u}_1 \mathbf{b}_{21}^T - \frac{1}{u_n} \mathbf{b}_{12} \mathbf{u}_1^T + \frac{b_{nn}}{u_n^2} \mathbf{u}_1 \mathbf{u}_1^T. \quad (11)$$

This can be verified by direct calculation, noting that $B\mathbf{e} = B^T \mathbf{e} = \mathbf{0}$ [63].

To illustrate the computation costs, we show in Table 1 some representative times on a linux office desktop using the scipy.sparse matrix package in python. For example, a 150×150 arena generates a probability transition matrix $\mathbf{P}$ of dimension $136,807 \times 136,807$ but has only 807,864 non-zero entries (at most 6 non-zero entries per row), or about $.004\% = 4 \times 10^{-5}$ of all the entries. Solving a system of equations with this matrix using restarted GMRES with an off-the-shelf preconditioner takes under 6 seconds. The time to setup the system can vary, partially because that part of the computation was not heavily optimized.

3.3. Distribution from Moments

In a later section below, we wish to extrapolate the behavior of multiple robots from the estimated average behavior for a single robot. To accomplish this we use the method of moments to fit an appropriate probability distribution for the hitting time. Based on the observed behavior from simulations, we observe that a gamma distribution with matching first and second moments yields a good fit to the observed distribution of hitting times. The gamma distribution is an appropriate distribution for a random quantity which is positive with no intrinsic upper limit. The first and second moments, i.e., the mean and variance, are obtained by direct calculation (6) and compared to those obtained by simulation on a 31×31 arena. [We show how well the gamma distribution fits simulation results both graphically in Figs. 1 & 2, and by applying a standard 1-sample Kolmogorov-Smirnov \(KS\) goodness-of-fit test in Table 2. It has been widely observed that fitness tests can be way overly strict on very large sample sizes \(e.g., \[75\]\). It has been suggested that one should apply the KS test to randomly selected smaller subsets, or simply note the small value of the KS statistic itself, which represents the maximum distance between the empirical and theoretical CDFs. Table 2](#)

shows result of the KS test on both an arbitrarily chosen smaller subset, with a p -value greater than .05 and on the full data set with a small KS statistic. The Quartile-Quartile diagram (Fig. 2 on the full data set shows how well they match and where they tend to differ. Both observations show that the gamma distribution is a plausible model for the hitting time.

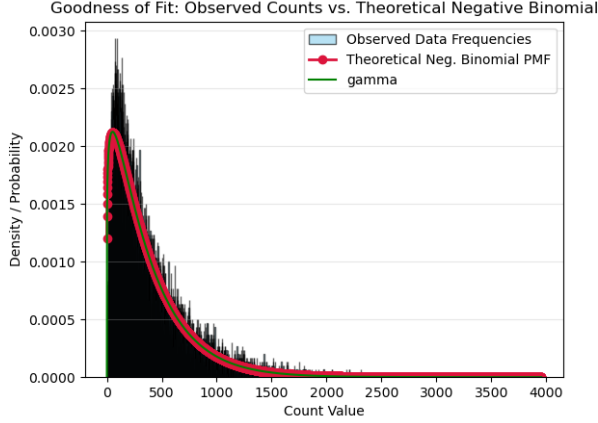


Figure 1: Fit between observed counts for a single robot on a 31×31 counts (black) and the negative binomial distribution (red) and gamma arena, starting in the lower left corner of the arena. The observed counts (green) distributions. The + signs mark, respectively, the 1%, 5%, 25%, 50% (median), 75%, 95%, and 99% percentiles, with a circle marking the median. The outliers are less than 1% of the total samples.

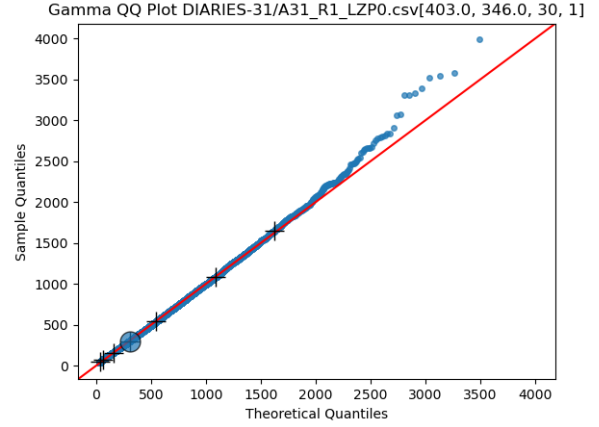


Figure 2: Quartile-quartile (QQ) plot showing match between observed

Table 2: Hitting time for a single robot placed at (0, 0) with an orientation of 0-degrees simulated on a 31×31 Arena. KS test compares Gamma distribution with previously calculated mean & variance to a selected subset of 300 simulation trials (see text), as suggested by [75].

N	mean $\pm$ std dev	KS stat	p -value	#trials	time(sec)	remarks
Th	402.658 $\pm$ 345.676	—	—	—	0.592	calculated
1	409.947 $\pm$ 315.872	.055	0.307	300	32.125	simulated
1	402.839 $\pm$ 346.989	.024	2e-16	30000	3175.356	simulated

We use the the gamma distribution to extrapolate the motion of a collection of robots under different circumstances as follows. The gamma distribution has a cumulative distribution function (CDF) [the probability the target will be reached in time less than or equal to T] as follows:

$$\text{CDF}(T) \stackrel{\text{def}}{=} \int_0^T \frac{\beta^\alpha}{\Gamma(\alpha)} \tau^{\alpha-1} e^{-\beta\tau} d\tau \quad (12)$$

with

$$\alpha = \frac{(\text{mean}(T))^2}{\text{var}(T)}, \quad \beta = \frac{\text{mean}(T)}{\text{var}(T)}, \quad (13)$$

Table 3: Hitting time metrics for the robot placed at (0, 0) with an orientation of 0-degrees simulated on a 31x31 Arena. This was tracked over multiple setups with different robot counts. Robots are randomly distributed except for the tracked robot, which is placed at the origin. Each robot follows a wait-next collision protocol.

N	mean $\pm$ std dev	wait%	KS stat	p value	#trials	time(sec)	remarks
Th	402.658 $\pm$ 345.676	LazyProb=0			–	0.592	calculated
Th	410.815 $\pm$ 352.742	LazyProb=0.02			–	0.550	calculated
15	406.268 $\pm$ 360.026	0.0150	0.049	0.443	3000	4696.365	simulated
16	396.167 $\pm$ 341.258	0.0163	0.065	0.152	3000	4873.889	simulated
17	399.635 $\pm$ 343.749	0.0172	0.031	0.928	3000	5233.454	simulated
18	395.899 $\pm$ 335.066	0.0181	0.060	0.217	3000	5473.713	simulated
19	398.609 $\pm$ 339.427	0.0192	0.053	0.337	3000	5830.134	simulated

where T is the random variable ($\mathbf{h} - \mathbf{h}_{\min}$ in this case), and α, β are the shape and rate parameters written in terms of the mean and variance of T . Here $\Gamma(\alpha)$ is the Gamma function. The hitting time cannot be less than the shortest path length $\mathbf{h}_{\min}$, hence we shift the Gamma distribution by this amount.

4. Verification of Properties via Simulations

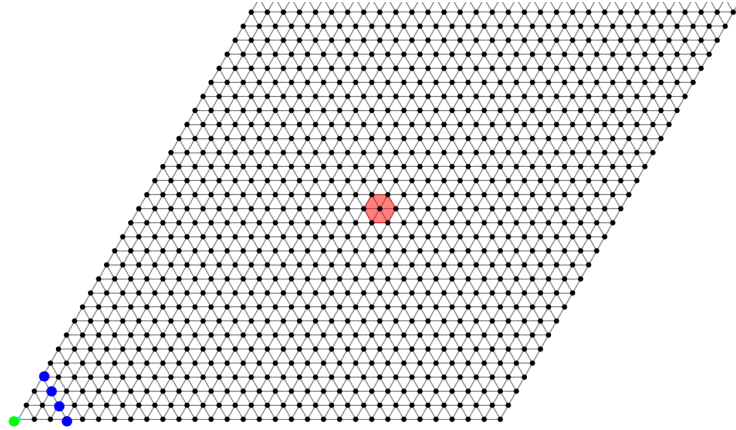


Figure 3: A typical $N \times N$ arena. A single robot enters at the corner in the lower left (green dot). The entrance point for multiple robots varies: see captions for tables 5 and 6. The black dots mark the possible locations for the robots and the edges denote the possible movements. The red nodes are nodes where the robot are within range to sense the goal, and hence proceed directly to be absorbed. The arena shown is for $N = 31$.

We illustrate a [typical](#) arena in Fig. 3. The arena is discretized into equilateral triangles as it allows a robot to traverse to six neighbors with equal length distances. We use triangles instead of a rectangular grid because the latter only has 4 possible directions of movement with equal length (north, south, east, west). Each state of the correlated random walk represents a combination of the location in the arena and the orientation of the robot, where

Table 4: Average hitting times for a single robot on a 61×61 arena with N other robots acting as obstacles, for N up to 500. The KS statistics & p -values are the results of fitness tests with a gamma distribution with parameters derived from the theoretical calculation with a laziness probability equal to $N/61^2$ where 61^2 is the arena size.

#obs	aver $\pm$ std dev	wait%	KS stat	p -val	#trials	time
Th	1835.44 $\pm$ 1617.28	LazyPr=0				2.332
Th	1886.04 $\pm$ 1661.97	LazyPr=100/61 ² =0.0269				2.306
Th	1939.52 $\pm$ 1709.18	LazyPr=200/61 ² =0.0537				2.270
Th	2119.89 $\pm$ 1868.43	LazyPr=500/61 ² =0.1344				2.660
1	1769.79 $\pm$ 1605.567	0.0	57.707	0.412	100	1221.078
100	1664.69 $\pm$ 1504.451	0.027	20.490	0.199	100	9538.046
200	2006.47 $\pm$ 1726.938	0.053	19.594	0.239	100	23066.567
500	1818.09 $\pm$ 1561.792	0.128	27.180	0.040	100	52590.992

the orientation is one of the six possible incoming directions. In this paper, we show computational predictions and simulations on 31×31 , 61×61 , and 101×101 arenas, all of the general form shown in Fig. 3, larger compared to those shown in [16]. In sec. 6 we show computatioanl results on a similar arena with some internal obstacles.

4.1. Average Times to Reach Nests

The CRW follows the model methodology laid out by previous papers on the same topic [68, 69]. Our configuration is based on that described in our earlier paper [16]. At each discrete time step, a robot selects one of 6 neighboring nodes to move to according to predefined probabilities, unless that position is already occupied (a “collision”). There are six different situations a robot can encounter when traversing an arena: I (has all six connected neighbors), CW (collision against an arena wall), PW (parallel to the wall), ASC (against sharp corner), AOC (against obtuse corner). We configured each robot to have a higher likelihood of moving forward than turning, since that is the most common practice. At each time step, the robot will randomly pick a neighboring state from the transition matrix to dictate its next move.

Additionally, absorbing nodes have been set around the target node, meaning that when a robot moves into these nodes, the transition probability to move into the target node is 1. We developed a simulation that follows the above rules. Data was collected off of the simulator and was compared to the Markov-chain computation approach; this methodology has been used in previous papers and will be the baseline of analysis for other configurations in this paper.

5. Multiple Robots and Obstacles

5.1. Problem Definition

In real-life situations, the area that a robot swarm traverses can be filled with objects and terrain that may impede movement. These are obstacles that can be identified by two categories: static and dynamic obstacles.

Static obstacles are entities that do not move throughout a simulation; instead, they block off different paths and states, making robots have to find alternative routes in order to reach their goal. If these static obstacle locations are fixed ahead of time, the transition matrix is set to have a 0% probability of moving to a state occupied by an obstacle.

Dynamic obstacles are defined as entities that move alongside the swarm, having a chance to interfere with a given robot's path, impeding its movement. As each robot in a given swarm moves independently of each other in a CRW policy, all of the other robots can be considered dynamic obstacles. We consider the case of a single tracked robot operating in an environment of other robots acting as obstacles. If the obstacles are distributed widely through the arena, then we can approximate the effect on the tracked robot by assuming there is a small probability that the tracked robot will need to stay put in its current position to avoid an obstacle. We can obtain a similar effect with a laziness probability. If we have a 61×61 arena with 100 obstacles, the probability that the position the tracked robot wishes to enter is occupied is approximately $100/61^2 = 0.0269$. Hence we obtain an approximation of the expected hitting time by carrying out the theoretical calculation using a laziness probability of 0.0269. This is illustrated for a few values of the number of obstacles in Table 4.

5.2. Approximating Expected First Arrival Time in a Swarm of Robots

We wish to compute an estimate of the time for the first of many robots to reach the nest. We consider N robots all following the same Correlated Random protocol independently. We assume all the robots follow the same distribution with identical expected hitting times. Let $p(t)$ be a gamma probability density function (PDF) for a single robot. The corresponding cumulative distribution function (CDF) is:

$$c(t) = \int_0^t p(\tau) d\tau \quad (14)$$

This gives the probability that a single robot reaches the nest by time t . Thus,

$$1 - c(t) = \Pr(\text{robot does not arrive by } t)$$

If we approximate a swarm of N independent robots avoiding collisions with N random walkers with an appropriate laziness value, then the probability no walker reaches the nest by time t would be $(1 - c(t))^N$. So the probability at least one walker reaches the nest by time t is

$$G_N(t) = 1 - (1 - c(t))^N. \quad (15)$$

Differentiating gives the PDF for the first arrival time:

$$g_N(t) = \frac{d}{dt}G_N(t) = N(1 - c(t))^{N-1}p(t)$$

Table 5: Average number of timesteps until the first robot among a group of N robots reaches the goal. The N robots are initially placed near the lower left corner in such a way they have equal shortest paths to the goal. The starting points for $N = 4$ are shown as blue dots in Fig. 3

N	mean± std dev	wait%	trials	time
31 × 31 arena				
Th	134.110± 95.792	Lazy=0	–	0.557
Th	71.359± 43.623	Lazy=0	–	0.584
4	138.538± 87.214	0.0144	3000	432.516
10	71.910± 39.773	0.0326	3000	559.705
61 × 61 arena				
Th	574.304±453.129	Lazy=0	–	2.381
Th	287.881±202.917	Lazy=0	–	2.266
4	610.708±417.604	0.0044	3000	4238.960
4	617.549±432.636	0.0043	3000	4296.415
10	345.091±191.271	0.0124	3000	5992.621
10	344.129±180.924	0.0128	3000	6007.022

Table 6: Arrival times for first of N robots initialized along the bottom and left edge of the arena.

1st-E: arena size 61 × 61, first among N robots

N	start	theory aver± std dev	simulation aver± std dev	wait%	simulate time(sec)	rem- arks
1	corner	1835.436±1617.285	1769.791±1605.567	0.0	1221.078	–
5	EDGE	484.149± 370.057	530.604± 334.109	0.008	1530.376	–
10	EDGE	292.981± 200.493	355.856± 186.439	0.020	2058.210	–
20	EDGE	188.941± 109.813	253.703± 106.116	0.037	2941.602	–
30	EDGE	151.465± 77.534	211.485± 88.018	0.046	3677.309	–
40	EDGE	131.756± 60.658	176.883± 67.104	0.048	4094.955	–
50	EDGE	119.470± 50.179	150.993± 60.022	0.048	4366.614	–
60	EDGE	111.025± 42.994	128.125± 51.609	0.045	13355.976	–
100	EDGE	93.255± 27.934	75.211± 26.903	0.047	4357.698	–

The expected first arrival time and corresponding standard deviation are (computed numerically)

$$\mu_N = \int_0^\infty t g_N(t) dt \quad \text{and} \quad \sigma_N = \sqrt{\int_0^\infty (t - \mu_N)^2 g_N(t) dt} \quad (16)$$

5.3. Adapting Model with Laziness Property

Given that robots and obstacles cannot inhabit the same position in the arena, a protocol is established for what should happen when a robot attempts to move into a cell already occupied. In this project we adopted a simple wait-next protocol: if a robot tries to move to a position occupied by a dynamic obstacle, it will stay in its current position for one cycle. When dynamic obstacles are introduced, the Markov chain model approach cannot be applied directly in this situation; the requirement for a fixed predefined transition matrix is not met. One workaround for this issue is find a model with a property that mimics the behavior of dynamic obstacles; we have found a "laziness" property to be a good candidate (see e.g., [52] & references therein).

We have defined laziness to be a property in which a robot has a set probability of not changing state, which follows the behavior of robots with a wait-next collision protocol. Mathematically, the transition matrix is modified to incorporate the possibility where a robot remains in its current state with probability p_l and transitions normally with probability $1 - p_l$. This property allows an isolated robot in the arena to delay state transitions, mirroring the effects of dynamic obstacles in multi-robot environments. Additionally, another important quality is that the transition matrix is static, meaning that methods to compute moments for mean and variance can be performed.

Using the computed Gamma Distribution with cumulative distribution function $c(t)$ (14), an estimated cumulative distribution for the probability that at least one robot in a swarm of n robots will reach the target in at most T steps is given by eq. (15). For example, a single walker in a corner of a 31×31 arena with hitting time moments of $\mu = 410$, $\sigma = 352$ indicates that the first of $N = 15$ walkers will reach the nest in $\mu_N \approx 64$ timesteps with a standard deviation of 29. This approximation yields a qualitative reflection of the performance of multiple robots while bypassing the need to simulate the system.

Table 7: This table shows the simulation results for the fastest robot in a swarm of varying sizes, 31x31 arena, filled all in one corner.

# Robots	Avg Timesteps	Std. Dev.
15	72.181	29.584
16	70.392	28.538
17	69.398	27.753
18	67.302	26.050

5.4. Statistical fit

For small n on a modest-sized arena, it is possible to compare this calculated distribution with empirical observations obtained from simulation. In this setup, robots collect items encountered while wandering through the arena and carry them to the goal node. **Figure 6 shows there is a good match** between the theoretical distribution and the empirical distribution observed from 300 runs over a 31×31 arena with 10 robots.

To compare the similarity in distributions between the multi-robot configuration and a fitted gamma distribution on the population, we performed a Kolmogorov-Smirnov (KS) test. The KS-test provides insight on the goodness-of-

Table 8: Calculated hitting times for a single robot with various levels of laziness, together with the predicted hitting time for the first of 18 robots to reach goal, on a 31×31 arena.

Laziness	$HT_1 \pm \text{StdDev}$	$HT_{18} \pm \text{StdDev}$
0.0%	402.658 ± 345.675	58.38 ± 24.18
0.5%	404.667 ± 347.415	58.56 ± 24.32
1.0%	406.695 ± 349.173	58.75 ± 24.47
1.5%	408.744 ± 350.948	58.94 ± 24.63
2.0%	410.814 ± 352.742	59.13 ± 24.78
2.5%	412.906 ± 354.554	59.32 ± 24.93
3.0%	415.019 ± 356.384	59.52 ± 25.09

fit of a distribution compared to another. Utilizing the KS-statistic, we ran a statistical analysis procedure to verify whether a set of samples from the data collected on a given configuration will match the fitted gamma distribution derived from the theoretical calculations. The null hypothesis is that the gamma is a good fit for the samples taken from the population and do not significantly differ. This procedure was then applied to each robot configuration. Given the sensitivity of fitness tests to the sample size [75], when there is a large set of trials we apply the KS test to the entire set and to a subset of 300 trials. The results appear in tables 2, 4. The results show p-values greater than 0.05, so the null hypothesis cannot be rejected. One concludes that the fitted gamma distribution is a plausible fit for the samples from simulation. Table 6 gives examples with more robots in the swarm, initialized along the left-most and bottom boundaries of the arena (a 61×61 arena similar to Fig. 3). In this case, the robots along the boundaries have different shortest path lengths to the goal, so if they were to operate alone, their distribution of hitting times would not be identical to one-another. This results in a more imperfect fit to the predictions obtained by the extrapolation of section 5.2.

After identifying that the fitted gamma distribution is appropriate for a multi-robot swarm, one can now conduct levels of fit between this distribution to the single-lazy robot simulation. In Table 5 7, this is the data collected for the single-robot simulation with laziness that matches the wait percentage for the multi-robots above.

Using (15), we can answer questions for much larger arenas for which the expense of simulation could be prohibitive. As an example, using the estimated distribution (15), we can quickly answer questions like: how many robots using a simple CRW would be needed to have an 80% chance to reach the goal in the center of a 150×150 arena within 500 time steps. By computing the estimated CDF on the 150×150 arena for various n , one can see from the resulting Figure 7 that approximately 200 robots would be needed.

6. Extensions

We give some further examples of analyses that can be accomplished by direct calculation without resorting to simulations.

6.1. Coverage Estimates

We can extract further information from the fundamental matrix $\mathbf{N}$ (4). For example we can quickly obtain estimates of how much of the domain will be traversed by an agent or collection of agents before reaching the goal. It can be shown [43] that the quantity

$$\mathbf{V}_{ij} = \mathbf{N}_{ij} / \mathbf{N}_{jj} \quad (17)$$

is the probability that the agent would pass node j at least once on the way from i to the goal [43]. This means that it is easy to rapidly compute estimates of which portions of an arena are more likely to be covered by agents starting in various initial positions. Fig. 4 shows the regions with a probability above certain threshold of being traversed, based on a quick computation of (17). A designer can use this information to select suitable starting points for their agent, or switch protocol when an agent enters a certain region. For example, in the scenario shown in Fig. 4, we show the coverage for an agent using a traditional correlated random walk with a high probability (48%) of continuing straight compared to the coverage if the agent has a bias (24%) of turning to the right in one case, or to the left in a second case, with tiny probability of turning the other way. One can see that the biased protocol leads to a greater tendency to follow closer to the walls while covering a greater fraction of the arena before reaching the goal.

6.2. Region with Irregularities

We illustrate the use of the linear algebra techniques proposed in this paper to a more irregular region, by adding a left-to-right barrier through the middle of the arena marked in black in figure 5. We consider the scenario in which robot agents are injected in the arena at one of the lower corners. The behavior when injected in an upper corner would be the same by symmetry. There are several properties that can be calculated without any simulation. We can also vary the protocol adopted by the robot agent to see how that affects the coverage of the arena or the time to reach goal. These quantities are given in Table 10.

The quantities represented by the matrix $\mathbf{N}$ (4) can be used to reveal estimate of the coverage of the arena by various robot agents starting from a variety of positions. We note from (5a) that $\mathbf{N}_{ij}$ is the average number of visits to a node j starting from node i before reaching a goal node. Taking advantage of the memoryless nature of random walks (correlated or not), it can be shown that the quantity (17) is the probability of passing node i on the way to node j . We illustrate this on an arena similar to the previous, but with a barrier placed along horizontal center line in the arena separating the lower part of the arena from the upper half. We leave an small opening between the lower and upper halves in the neighborhood of the goal located in the center of the whole arena.

7. Conclusion

This paper has established a foundation for which Markov chains and graph algorithms can be utilized to obtain quick estimates of different properties of robot swarms, such as hitting times, without having to run simulations. Throughout this project, we have explored the possibility of dynamic and static obstacles being introduced in the simulation, and how the models can be adjusted to fit them. The static laziness parameter is imperfect in matching the nuance of dynamic robots as obstacles; however, it yields results that are consistent and similar to multi-robot swarms. After obtaining the distribution, one can then perform necessary analysis to see which configuration of robot swarm is needed in their project.

As Markov-chains haven't been extensively used in robot swarming, we propose that this could be a very promising direction to explore when it comes to optimizing robot swarm designs. Many local variations of robot movement policies are based off of a random walk approach, meaning that the approach outlined above can generate rapid approximations for more complex systems. Future work will outline different areas and directions to continue this research.

7.1. Future Work

The proposed methods have some limitations due to its simplification of how real robots operate. Using a static laziness parameter to approximate dynamic collisions is not perfect, but does yield results that are qualitatively consistent. We have modeled the time for one particulate robot to reach the goal. This work should be thought of as preliminary to a model for tracking when one of many robots reach the goal, a future direction.

Additionally, as the results for a laziness robot mapping may not be appropriate, further analysis and research will have to be done on potential conditions on why this difference occurs, especially with larger arena sizes. Future work will include evaluating the effects when the arena is more crowded and alternative statistical tests such as Kullback–Leibler divergence and Earth Mover's Distance to compare distributions.

In this paper, we have shown that graph theory can offer a rich set of tools with which to analyze a multi-robot system and predict its global motion behavior without having to run expensive simulations. This suggests that further development of graph-based models for swarm robotics would be a very promising direction to pursue.

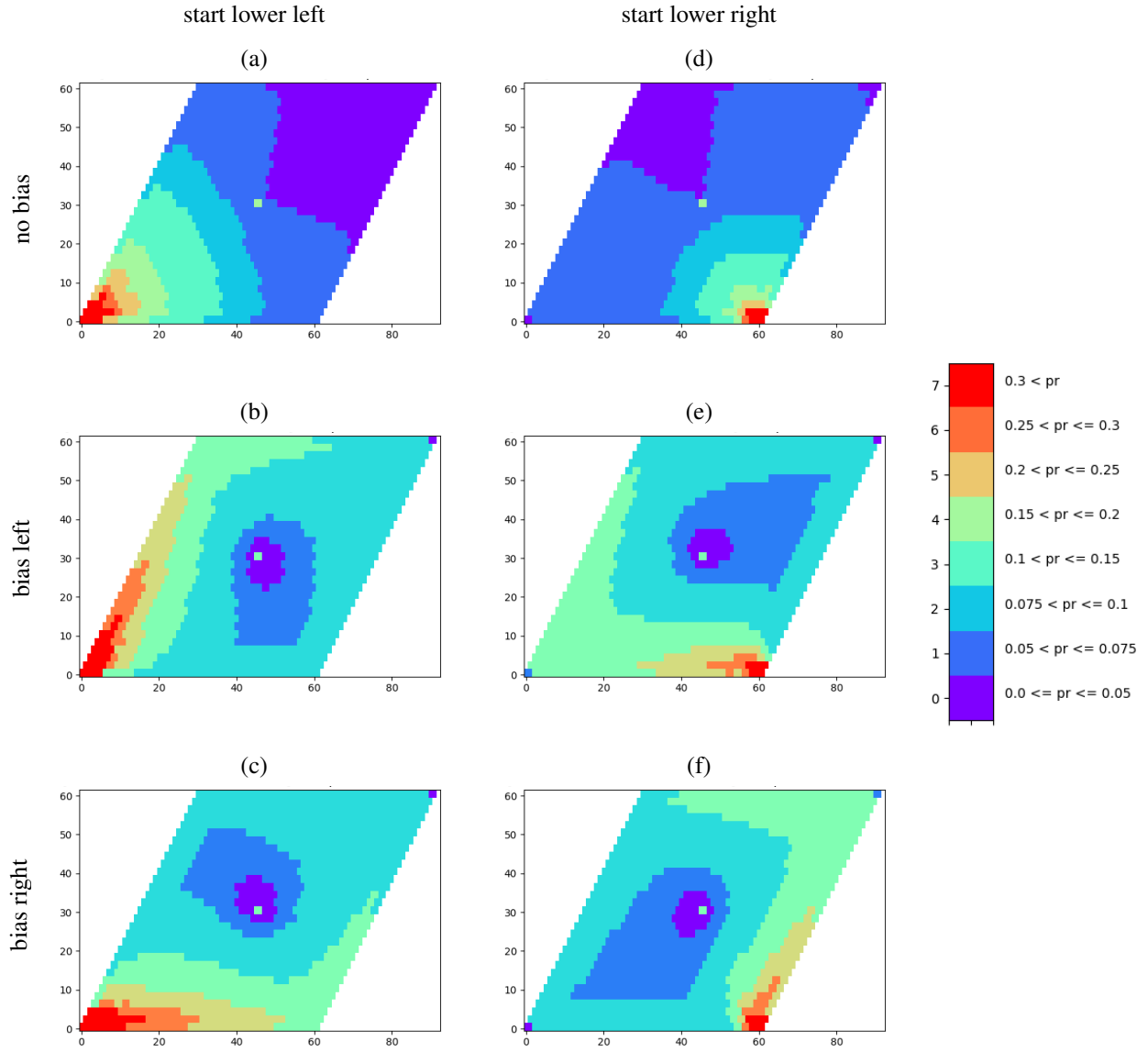


Figure 4: Spread of robots with a single goal in the middle.

Table 9: Aggregate behavior for the arena with a single goal in the middle, shown in Fig. 4 with robots injected from the lower left or lower right, with optional bias to turn to the left or right. The properties shown are the expected time to reach goal (with standard deviation), the fraction of the arena with a given probability of being visited by a robot, and the expected time for the first of 18 robots to reach the goal.

Figure	start	bias	hitting $\pm$ std_dev	$p > .075$	$p > .1$	ave ₁₈ $\pm$ std ₁₈
4(a)	left	none	402.66 $\pm$ 345.68	35	22	58.38 $\pm$ 24.18
4(c)	left	left	808.01 $\pm$ 722.56	97	84	88.28 $\pm$ 50.56
4(e)	left	right	808.01 $\pm$ 722.56	97	84	88.28 $\pm$ 50.56
4(b)	right	none	342.46 $\pm$ 341.78	20	09	42.46 $\pm$ 13.57
4(d)	right	left	771.74 $\pm$ 723.75	98	78	76.20 $\pm$ 43.41
4(f)	right	right	769.70 $\pm$ 723.60	98	77	75.62 $\pm$ 43.04

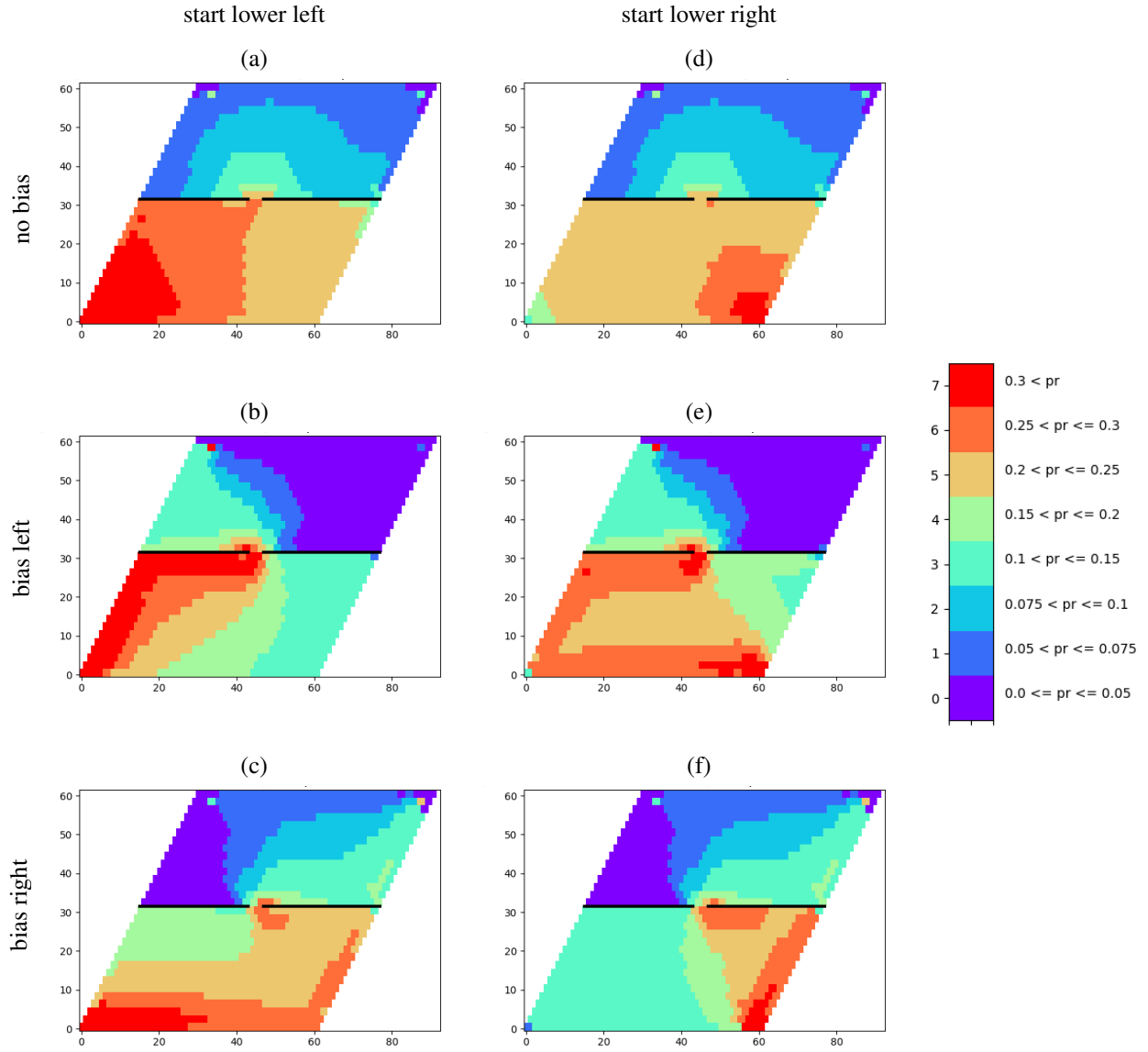


Figure 5: Spread of robots two goals and a barrier across the middle.

Table 10: Aggregate behavior for the arena two goals and a barrier across the middle. shown in Fig. 5 with robots injected from the lower left or lower right, with optional bias to turn to the left or right. The properties shown are the expected time to reach goal (with standard deviation), the fraction of the arena with a given probability of being visited by a robot, and the expected time for the first of 18 robots to reach the goal.

Figure	start	bias	hitting $\pm$ std_dev	p>.075	%:p>.1	ave ₁₈ $\pm$ std ₁₈
5(a)	left	none	1077.70 $\pm$ 873.70	78	58	142.11 $\pm$ 82.26
5(c)	left	left	868.90 $\pm$ 772.11	68	65	94.85 $\pm$ 55.51
5(e)	left	right	950.12 $\pm$ 704.01	76	66	156.15 $\pm$ 81.09
5(b)	right	none	984.19 $\pm$ 868.63	78	58	106.60 $\pm$ 64.52
5(d)	right	left	967.95 $\pm$ 778.64	68	65	131.83 $\pm$ 74.13
5(f)	right	right	738.88 $\pm$ 680.04	75	66	77.16 $\pm$ 43.06

References

- [1] I. Wagner, Y. Altshuler, V. Yanovski, A. Bruckstein, Cooperative cleaners: A study in ant robotics, *The International Journal of Robotics Research (IJRR)* 27 (1) (2008) 127–151.
- [2] J. P. Lancaster, D. A. Gustafson, Predicting the behavior of robotic swarms in search and tag tasks, *Procedia Computer Science* 20 (2013) 77–82, complex Adaptive Systems. doi:<https://doi.org/10.1016/j.procs.2013.09.242>.
URL <https://www.sciencedirect.com/science/article/pii/S1877050913010429>
- [3] H. Hamann, Modeling swarm systems and formal design methods, in: *Swarm Robotics: A Formal Approach*, Springer, Berlin/Heidelberg, Germany, 2018, pp. 95–127.
- [4] M. Kegeleirs, D. G. Ramos, M. Birattari, Random walk exploration for swarm mapping, in: *Proc. Towards Autonomous Robotic Systems: 20th Annual Conference, TAROS 2019, Part II*, Springer-Verlag, Berlin, Heidelberg, 2019, pp. 211–222.
- [5] C. Pincioli, V. Trianni, R. O’Grady, G. Pini, A. Brutschy, M. Brambilla, N. Mathews, E. Ferrante, G. Di Caro, F. Ducatelle, et al., ARGoS: a modular, multi-engine simulator for heterogeneous swarm robotics, in: *IEEE/RSJ Intel Conf Intelligent Robots and Systems*, IEEE, New York City, 2011, pp. 5027–5034.
- [6] B. Pang, Y. Song, C. Zhang, H. Wang, R. Yang, A swarm robotic exploration strategy based on an improved random walk method, *Journal of Robotics* 2019 (Article ID 6914212) (2019) 9.
- [7] F. Martinez, E. Jacinto, D. Acero, Brownian motion as exploration strategy for autonomous swarm robots, in: *2012 IEEE Int’l Conf. on Robotics and Biomimetics (ROBIO)*, IEEE, New York City, NY, 2012, pp. 2375–2380.
- [8] R. Fujisawa, S. Dobata, Lévy walk enhances efficiency of group foraging in pheromone-communicating swarm robots, in: *IEEE/SICE Int’l Symposium on System Integration, SII 2013*, IEEE, New York City, NY, 2013, pp. 808–813.
- [9] D. A. Levin, Y. Peres, E. L. Wilmer, *Markov Chains and Mixing Times*, Vol. 107, American Mathematical Soc., Providence, RI, 2017.
- [10] L. Lovász, Random walks on graphs, *Combinatorics* 2 (1-46) (1993) 4.
- [11] N. Masuda, M. A. Porter, R. Lambiotte, Random walks and diffusion on networks, *Physics reports* 716 (2017) 1–58.
- [12] R. Nain, K. Sen, Transition probability matrices for correlated random walks, *Journal of Applied Probability* 17 (1) (1980) 253–258.
- [13] M. Jeong, J. Harwell, M. Gini, Analysis of exploration in swarm robotic systems, in: *IAS-16*, Springer, Berlin/Heidelberg, Germany, 2021, pp. 445–457.
- [14] J. Harwell, M. Gini, Broadening applicability of swarm-robotic foraging through constraint relaxation, in: *2018 IEEE Int’l Conf. on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAN)*, IEEE, New York City, NY, 2018, pp. 116–122.
- [15] C. S. Patlak, Random Walk with Persistence and External Bias: A Mathematical Contribution to the Study of Orientation of Organisms, University of Chicago, Committee on Mathematical Biology, 1953.
- [16] J. Cadavez, D. Boley, Estimating robot swarm properties by direct calculations instead of simulation, in: *IAS-19 (19th International Conference on Intelligent Autonomous Systems)*, 2025.
- [17] D. Zhou, J. Huang, B. Schölkopf, Learning from labeled and unlabeled data on a directed graph, in: *Proc. 22nd Int’l Conf. Machine Learning*, ACM, New York City, NY, 2005, pp. 1041–1048.
- [18] M. B. Cohen, J. Kelner, J. Peebles, R. Peng, A. Sidford, A. Vladu, Faster algorithms for computing the stationary distribution, simulating random walks, and more, in: *IEEE 57th Annual Symp. on Found. Comput. Sci. (FOCS)*, IEEE, New York City, NY, 2016, pp. 583–592.
- [19] F. Chung, *Spectral Graph Theory*, Am. Math. Soc., Providence, RI, 1997.
- [20] W. E. Donath, A. J. Hoffman, Lower bounds for the partitioning of graphs, *IBM J. of Res. and Dev.* 17 (1973) 410–425.
- [21] M. Fiedler, A property of eigenvectors of nonnegative symmetric matrices and its applications to graph theory, *Czechoslovak Math. J.* 25 (100) (1975) 619–633.
- [22] U. von Luxburg, A tutorial on spectral clustering, *Statistics and Computing* 17 (4) (2007) 395–416, max Planck Institute for Biological Cybernetics. Technical Report No. TR-149.
- [23] N. Alon, Eigenvalues and expanders, *Combinatorica* 6 (2) (1986) 83–96.

- [24] F. Fouss, A. Pirotte, J. michele Renders, M. Saelens, Random-walk computation of similarities between nodes of a graph with application to collaborative recommendation, *IEEE Trans. on Knowledge and Data Engineering* 19 (2007) 355–369.
- [25] J. Shi, J. Malik, Normalised cuts and image segmentation, *IEEE Trans. Pattern Analysis and Machine Intelligence* 22 (8) (2000) 888–905.
- [26] R. A. Brualdi, H. J. Ryser, *Combinatorial Matrix Theory*, Cambridge Univ. Press, Cambridge, UK, 1991.
- [27] P. Chebotarev, R. Agaev, Forest matrices around the laplacian matrix, *Lin. Alg. and its Appl.* 356 (1-3) (2002) 253–274.
- [28] F. Chung, Laplacians and the Cheeger inequality for directed graphs, *Annals of Combinatorics* 9 (1) (2005) 1–19.
- [29] J. G. Kemeny, J. L. Snell, *Finite Markov chains (with a new appendix “Generalization of a fundamental matrix”)*, Springer, Berlin/Heidelberg, Germany, 1983.
- [30] D. Aldous, J. A. Fill, Reversible markov chains and random walks on graphs, <http://www.stat.berkeley.edu/users/aldous/RWG/book.html> (2014).
- [31] Y. Altshuler, A. Bruckstein, Static and expanding grid coverage with ant robots: Complexity results, *Theoretical Computer Science (TCS)* 412 (35) (2011) 4661–4674.
- [32] M. Sardar, D. Das, S. Mukhopadhyaya, Grid exploration by a swarm of autonomous robots with minimum repetitions, *Theoretical Computer Science* 933 (2022) 67–87.
- [33] B. Pang, Y. Song, C. Zhang, R. Yang, Effect of random walk methods on searching efficiency in swarm robots for area exploration, *Applied Intelligence* 51 (7) (2021) 5189–5199.
- [34] U. Feige, A tight upper bound on the cover time for random walks on graphs, *Random structures and algorithms* 6 (1) (1995) 51–54.
- [35] Y. Li, Z.-L. Zhang, Digraph Laplacian and the degree of asymmetry, *Internet Mathematics* 8 (4) (2012) 381–401.
- [36] Z. Zhang, A. Julaiti, B. Hou, H. Zhang, G. Chen, Mean first-passage time for random walks on undirected networks, *The European Physical Journal B* 84 (4) (2011) 691–697.
- [37] S. Ikeda, I. Kubo, M. Yamashita, The hitting and cover times of random walks on finite graphs using local degree information, *Theoretical Computer Science* 410 (1) (2009) 94–100.
- [38] G. Brightwell, P. Winkler, Maximum hitting time for random walks on graphs, *Random Structures & Algorithms* 1 (3) (1990) 263–276.
- [39] S. I. Maiti, J. Sarkar, Symmetric walks on paths and cycles, *Mathematics Magazine* 92 (4) (2019) 252–268.
- [40] R. Bapat, On the first passage time of a simple random walk on a tree, *Statistics & probability letters* 81 (10) (2011) 1552–1558.
- [41] R. Patel, A. Carron, F. Bullo, The hitting time of multiple random walks, *SIAM Journal on Matrix Analysis and Applications* 37 (3) (2016) 933–954.
- [42] N. Rivera, T. Sauerwald, J. Sylvester, Multiple random walks on graphs: mixing few to cover many, *arXiv preprint arXiv:2011.07893* (2020).
- [43] G. Golnari, Z.-L. Zhang, D. Boley, Markov fundamental tensor and its applications to network analysis, *Linear Algebra and its Applications* 564 (2019) 126–158.
- [44] C. M. Grinstead, J. L. Snell, *Introduction to Probability*, Amer Math Soc., Boston, MA, 1997.
- [45] D. Boley, G. Ranjan, Z.-L. Zhang, Commute times for a directed graph using an asymmetric laplacian, *Linear Algebra and its Applications* 435 (2) (2011) 224–242.
- [46] N. Alon, C. Avin, M. Koucký, G. Kozma, Z. Lotker, M. R. Tuttle, Many random walks are faster than one, *Combinatorics, Probability and Computing* 20 (4) (2011) 481–502.
- [47] P. Caputo, M. Quattropani, Stationary distribution and cover time of sparse directed configuration models, *Probability Theory and Related Fields* 178 (3) (2020) 1011–1066.
- [48] K. Efremenko, O. Reingold, How well do random walks parallelize?, in: I. Dinur, K. Jansen, J. Naor, J. Rolim (Eds.), *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX RANDOM 2009)*, Lecture Notes in Computer Science, Vol. 5687, Springer, Berlin, Heidelberg, 2009.
- [49] R. Elsässer, T. Sauerwald, Tight bounds for the cover time of multiple random walks, *Theo. Comp. Sci.* 412 (24) (2011) 2623–2641.
- [50] M. T. Barlow, Y. Peres, P. Sousi, Collisions of random walks, *Annales de l’IHP Probabilités et statistiques* 48 (4) (2012) 922–946.
- [51] A. Gaudillière, Collision probability for random trajectories in two dimensions, *Stoch. proc. & their appl.* 119 (3) (2009) 775–810.

- [52] J. Harwell, A. Sylvester, M. Gini, An empirical characterization of ODE models of swarm behaviors in common foraging scenarios, *Autonomous Robots* (2023). doi:10.1007/s10514-023-10121-9.
URL <https://doi.org/10.1007/s10514-023-10121-9>
- [53] Q. Dong, L. Cui, First hitting time distributions for brownian motion and regions with piecewise linear boundaries, *Methodology and Computing in Applied Probability* 21 (2019) 1–23.
- [54] M. Ding, K. Y. Szeto, First-passage time distribution for random walks on complex networks using inverse laplace transform and mean-field approximation, *arXiv preprint arXiv:1812.05598* (2018).
- [55] H. W. Lau, K. Y. Szeto, Asymptotic analysis of first passage time in complex networks, *EPL (Europhysics Letters)* 90 (4) (2010) 40005.
- [56] H. Chen, The generating functions of hitting times for random walk on trees, *Statistics & probability letters* 77 (15) (2007) 1574–1579.
- [57] N. Zlatanov, L. Kocarev, Random walks on networks: Cumulative distribution of cover time, *Physical Review E* 80 (4) (2009) 041102.
- [58] J. J. Hunter, The distribution of mixing times in Markov chains, *Asia-Pacific Journal of Operational Research* 30 (01) (2013) 1250045.
- [59] A. H. Foss, R. B. Lehoucq, W. Z. Stuart, J. D. Tucker, J. W. Berry, A deterministic hitting-time moment approach to seed-set expansion over a graph, *arXiv preprint arXiv:2011.09544* (2020).
- [60] T. Dayar, W. J. Stewart, On the effects of using the grassmann–taksar–heyman method in iterative aggregation–disaggregation, *SIAM Journal on Scientific Computing* 17 (1) (1996) 287–303.
- [61] T. Dayar, N. Akar, Computing moments of first passage times to a subset of states in markov chains, *SIAM Journal on Matrix Analysis and Applications* 27 (2) (2005) 396–412.
- [62] W. K. Grassmann, M. I. Taksar, D. P. Heyman, Regenerative analysis and steady state distributions for markov chains, *Operations Research* 33 (5) (1985) 1107–1116.
- [63] D. Boley, On fast computation of directed graph laplacian pseudo-inverse, *Linear Algebra and its Applications* 623 (2021) 128–148.
- [64] Q. Lu, G. M. Fricke, J. C. Ericksen, M. E. Moses, Swarm foraging review: Closing the gap between proof and practice, *Current Robotics Reports* 1 (2020) 1–11.
- [65] A. Aggarwal, D. Gupta, W. Vining, G. Fricke, M. Moses, Ignorance is not bliss: an analysis of central-place foraging algorithms, in: *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, New York City, NY, 2019, pp. 6510–6517.
URL doi:10.1109/IROS40897.2019.8967665
- [66] S. Cannon, J. Daymude, C. Gokmen, D. Randall, A. Richa, A local stochastic algorithm for separation in heterogeneous self-organizing particle systems, <https://arxiv.org/abs/1805.04599> (2019).
- [67] S. Oh, D. Randall, A. W. Richa, Foraging in particle systems via self-induced phase changes, <https://arxiv.org/abs/2208.10720> (2022).
- [68] Y. Zhang, D. Boley, J. Harwell, M. Gini, A correlated random walk model to rapidly approximate hitting time distributions in multi-robot systems, in: *Intelligent Autonomous Systems 17, Proc. 17th Int’l Conf IAS-17*, Springer, Berlin/Heidelberg, Germany, 2022, pp. 724–736.
- [69] D. Boley, M. Gini, Y. Zhang, How do robot swarms behave? what graphs can tell us, in: *Autonomous Robots and Multirobot Systems (ARMS) 2023 Workshop at AAMAS 2023*, 2023.
URL <https://u.cs.biu.ac.il/~agmon/arms2023/program.html>
- [70] W. Joubert, On the convergence behavior of the restarted GMRES algorithm for solving nonsymmetric linear systems, *Num. Lin. Alg. Appl.* 1 (5) (1994) 427–447.
- [71] R. B. Morgan, GMRES with deflated restarting, *SIAM J Sci Comput* 24 (1) (2002) 20–37.
- [72] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd Edition, SIAM, 2003.
- [73] M. B. Cohen, J. A. Kelner, J. L. T. Peebles, R. Peng, A. B. Rao, A. Sidford, A. V. Vladu, Almost-linear-time algorithms for markov chains and new spectral primitives for directed graphs, in: *STOC 2017*, 2017.
- [74] G. W. Stewart, Simultaneous iteration for computing invariant subspaces of non-Hermitian matrices, *Numer. Math.* 25 (1976) 123–136.
- [75] D. Bergh, Sample size and chi-squared test of fit - a comparison between a random sample approach and a chi-square value adjustment method using swedish adolescent data, in: *Pacific Rim Objective Measurement Symposium (PROMS) 2014 Conference Proceedings*, 2015, pp. 197–211. doi:https://doi.org/10.1007/978-3-662-47490-7_15.

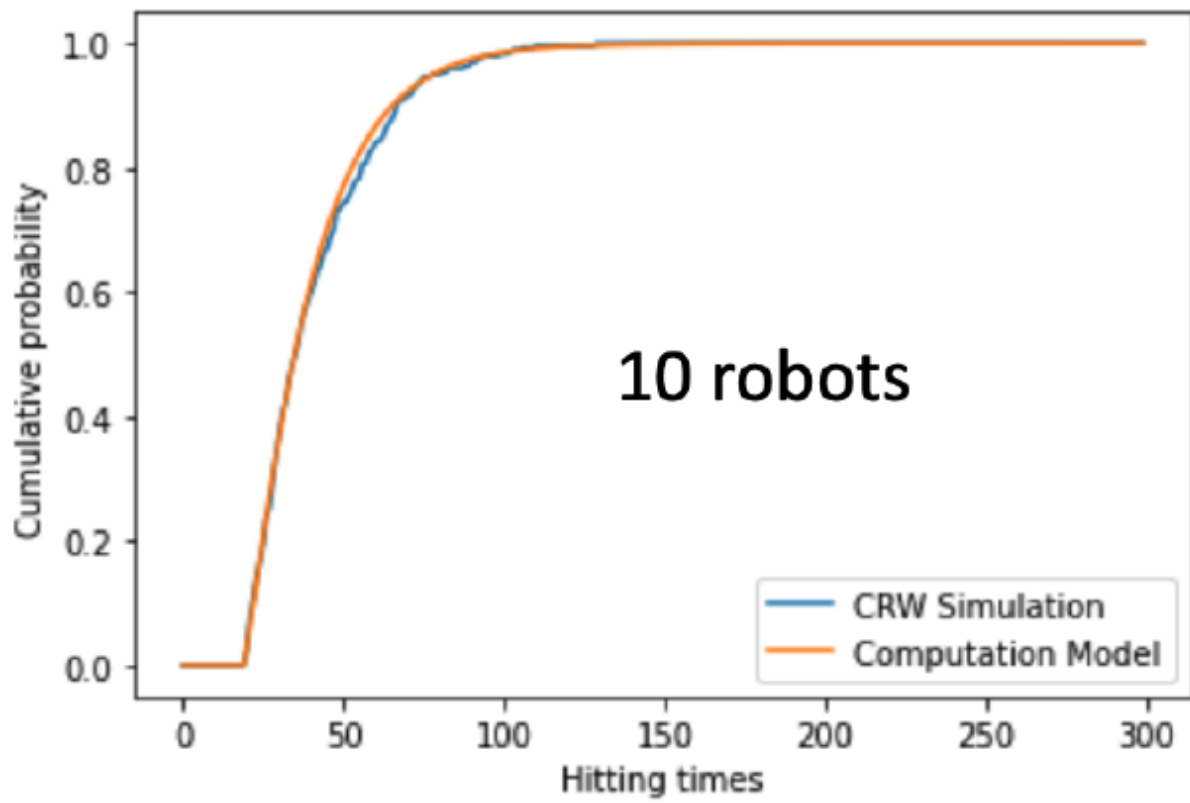


Figure 6: Times for the first of 10 robots to reach the goal node in the middle of a 31×31 arena: simulated vs calculated.

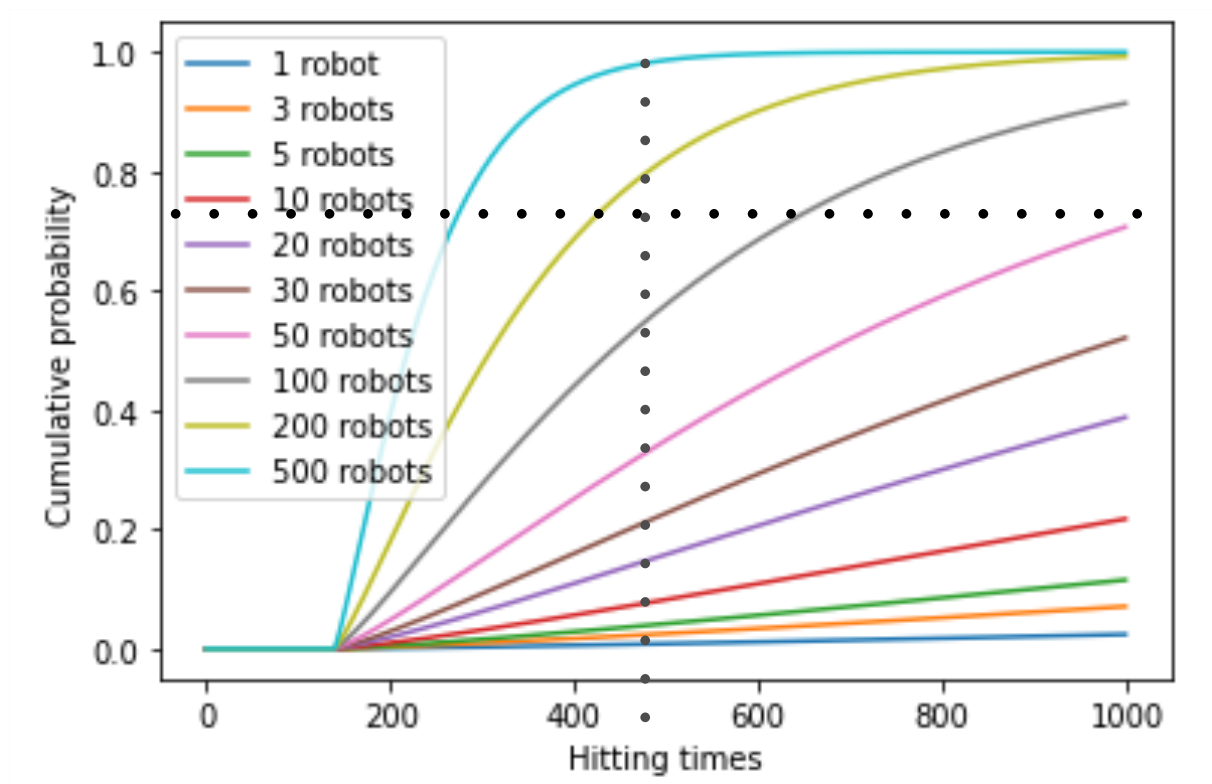


Figure 7: Calculated CDFs of hitting times for the first of many robots to reach the goal in the center of a 150×150 arena. The horizontal dotted line marked the 80% level for the given CDF.